2. Alternative Solutions and Trade-off Analysis

HackTheAPI // OWASP API Security Training & Assessment Platform

---

■ 1. Executive Summary & Design Decision Context

During the design and architectural phase of **HackTheAPI**, several design alternatives were evaluated across five critical architectural pillars:

- **AI Assistant Integration Strategy**
- **Lab Evaluation & Verification Engine**
- **Lab Workspace Containerization**
- **User Verification & Password Security**
- **Frontend Application Framework**

This document details the trade-off analysis, evaluation criteria, and technical rationale that led to the final system architecture.

■■ 2. Architectural Trade-off Analyses

Pillar 1: AI Assistant Integration Strategy

Evaluation Criteria	Option A: External Cloud LLM APIs (OpenAI GPT-4 / Gemini API)	Option B: Static Hardcoded Rule Engine	Option C: Local Ollama LLM Service (`qwen2.5-coder`) [SELECTED]
Data Privacy	■ Sends student query & lab context to cloud servers	■ 100% local, no external data transmission	**■ 100% local (`127.0.0.1:11434`), zero data leakage**
Operating Cost	■ Recurring API token cost per request	■ Zero API cost	**■ Zero API cost, open-source model execution**
Internet Dependency	■ Fails when offline or network drops	■ Works offline	**■ Works 100% offline on local workstation**
Response Quality	■ High general knowledge	■ Rigid, low adaptability to student queries	**■ High coding & security reasoning capability**
Screen Context Reader	■■ High latency sending full page state to API	■ Unable to interpret dynamic JSON workbench state	**■ Instant local prompt injection with page state**

Selection Rationale: Option C (**Local Ollama `qwen2.5-coder`**) was chosen because it guarantees complete student data privacy, zero recurring cloud API costs, full offline capability, and instant local execution with dynamic frontend workbench context.

Pillar 2: Lab Evaluation & Verification Mechanism

Evaluation Criteria	Option A: Client-Side Flag Submission (CTF Style)	Option B: Server-Side Dual Oracle Verification Engine [SELECTED]
Tamper Resistance	■ Vulnerable to client-side JS manipulation or flag hardcoding	■ **Tamper-proof; evaluates request payload & backend DB state**
Evaluation Depth	■ Only checks string comparison	■ **Evaluates physical database state mutations (e.g. user deletion)**
Realism	■ Unrealistic (real APIs do not give CTF flags)	■ **Simulates real enterprise API security auditing**
Automation	■ Simple static check	■ **Automated dynamic backend assertion checks**

Selection Rationale: Option B (**Server-Side Dual Oracle Engine**) was selected to eliminate client-side flag tampering and provide authentic, state-based verification of vulnerability exploitation.

Pillar 3: Lab Workspace Containerization

Evaluation Criteria	Option A: Shared Single Server Backend	Option B: Full Virtual Machines (VMware/Vagrant)	Option C: Per-User Isolated Docker Containers [SELECTED]
Resource Overhead	■ Very low RAM/CPU	■ Extremely heavy (GBs RAM per student)	■ **Lightweight (MBs RAM, instant startup)**
Tenant Isolation	■ High risk of cross-student collision	■ Complete VM isolation	■ **Isolated Docker bridge networks per session**
Reset Speed	■ Complex DB rollback	■ Slow VM snapshot restore	■ **Instant container restart/recreation**

Selection Rationale: Option C (**Docker Container Workspaces**) balances lightweight resource consumption with strong network and database isolation per user session.

Pillar 4: Email Verification & Security

Evaluation Criteria	Option A: Console Dummy Log Email	Option B: Dedicated Gmail SMTP with 3-Attempt Rate Limit [SELECTED]
Realism	■ Non-functional for real users	■ **Delivers actual verification codes to student inbox**
Brute-Force Security	■ Unlimited local tries	■ **Strict 3-attempt token lockout and invalidation**

Selection Rationale: Option B provides realistic email delivery via **`smtp.gmail.com:587`** with strict 3-attempt security thresholds.

Pillar 5: Frontend Architecture

Evaluation Criteria	Option A: Django Server-Side Rendered (HTML/Jinja)	Option B: React 18 Single Page Application (SPA) [SELECTED]
User Experience	■ Full page reloads on every tab click	■ **Fluid, zero-reload SPA navigation**
AI Panel Integration	■ Awkward drawer overlay	■ **VS Code Copilot style drawer with live reflow**
State Sync	■ Static DOM rendering	■ **Real-time `window.__CURRENT_LAB_WORKBENCH_STATE__` sync**

Selection Rationale: Option B (**React 18 SPA with Vite**) enables seamless state management, real-time payload workbench updates, and smooth AI drawer reflow.

■ 3. Summary Decision Matrix

```
Selected Stack:
- Frontend: React 18 + Vite + Tailwind CSS + Lucide Icons
- Backend: Django 5.1 REST Framework + SQLite/PostgreSQL
- AI Engine: Local Ollama LLM (`qwen2.5-coder`) on port 11434
- Labs Engine: Isolated Docker Containers (Ports 8101-8112)
- Auth & Security: JWT Bearer Tokens + Gmail SMTP (3-Attempt Rate Limit)
```